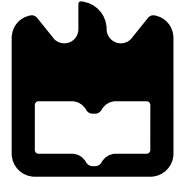


universidade de aveiro



deti

departamento de eletrónica,
telecomunicações e informática

Distributed Systems

Consistency and Replication

Eurico Pedrosa <efp@ua.pt>

2nd semester - 2025'26

Consistency and Replication in Distributed Systems

Consistency and Replication

Replication is a fundamental technique in distributed systems involving the maintenance of multiple copies of data across different nodes.

- It is primarily employed to achieve two key system goals:
 - **Reliability**: If one replica fails, the system can continue operating using others
 - **Performance**: Improving system scalability regarding size and geographical area by placing data closer to users and balancing the workload
- The central challenge is maintaining **consistency**: ensuring that when one copy of a data item is updated, the change is propagated to all other replicas so that they remain the same
- Informally, a data store is consistent if it behaves as if it were a single-machine system, even though it is physically distributed

Performance vs Consistency

Replication and **caching** are widely used to solve scalability problems.

- **Tight Consistency:** Requires updates to be performed on all copies as a single atomic operation (synchronous replication).
- **Implementation Hardship:** Realizing atomicity across a wide-area network is difficult because replicas must reach agreement on the global ordering of operations.
- **Trade-off:**
 - **Strong consistency** requires global synchronization, costly in latency and performance
 - **Relaxing consistency** can improve performance by avoiding instantaneous sync.

The extent to which consistency can be relaxed depends entirely on the specific access patterns and requirements of the application.

Data-Centric Consistency Models

Data-Centric Consistency Models

A consistency model acts as a contract between processes and a data store, specifying the rules that processes must obey to ensure the store works correctly.

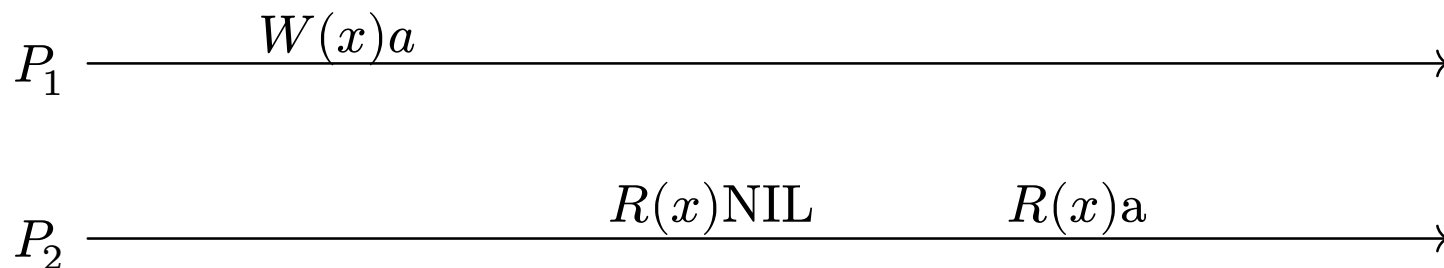
- **Concept:** A read operation on a data item is generally expected to return a value showing the results of the “last” write operation
- **Trade-off:** Models with major restrictions (strong consistency) are easier to use for developers but perform poorly, while models with minor restrictions (weak consistency) offer better performance but are harder to use.
- **Data Store:** A logical collection of data that may be physically distributed and replicated across multiple machines.

Sequential Consistency

A data store is **sequentially consistent** if the result of any execution is the same as if the operations by all processes were executed in some sequential order.

- **Program Order:** The operations of each individual process must appear in the global sequence in the order specified by its program.
- **Global Interleaving:** All processes must see the **same** interleaving of read and write operations, regardless of absolute time.
- **Example:** If process P_1 writes 'a' and P_2 writes 'b', all other processes must agree on whether 'a' happened before 'b' or vice versa.

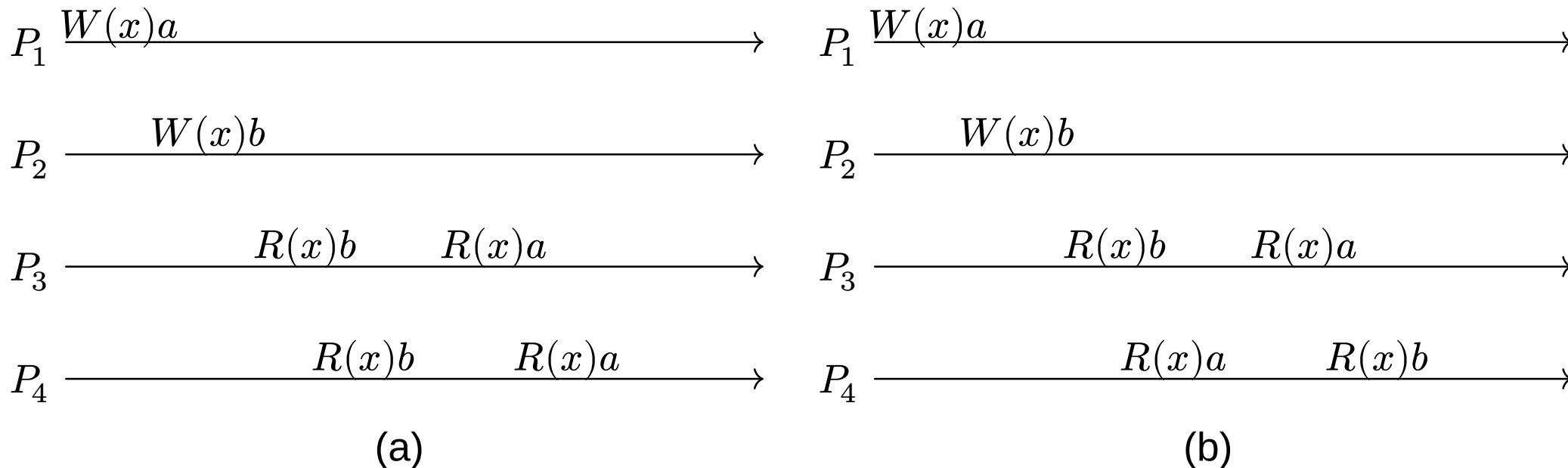
Sequential Consistency: Example



- **Notation**

- ▶ Time axis is drawn horizontally
- ▶ $R_{i(x)}a$ denotes that process P_i writes value a to item x
- ▶ $R_{i(x)}b$ denotes that process P_i reads x and returns the value b
- ▶ each data item has an initial value NIL
- ▶ i can be omitted if there is no confusion about with process is accessing the data

Sequential Consistency: Example



Linearizability

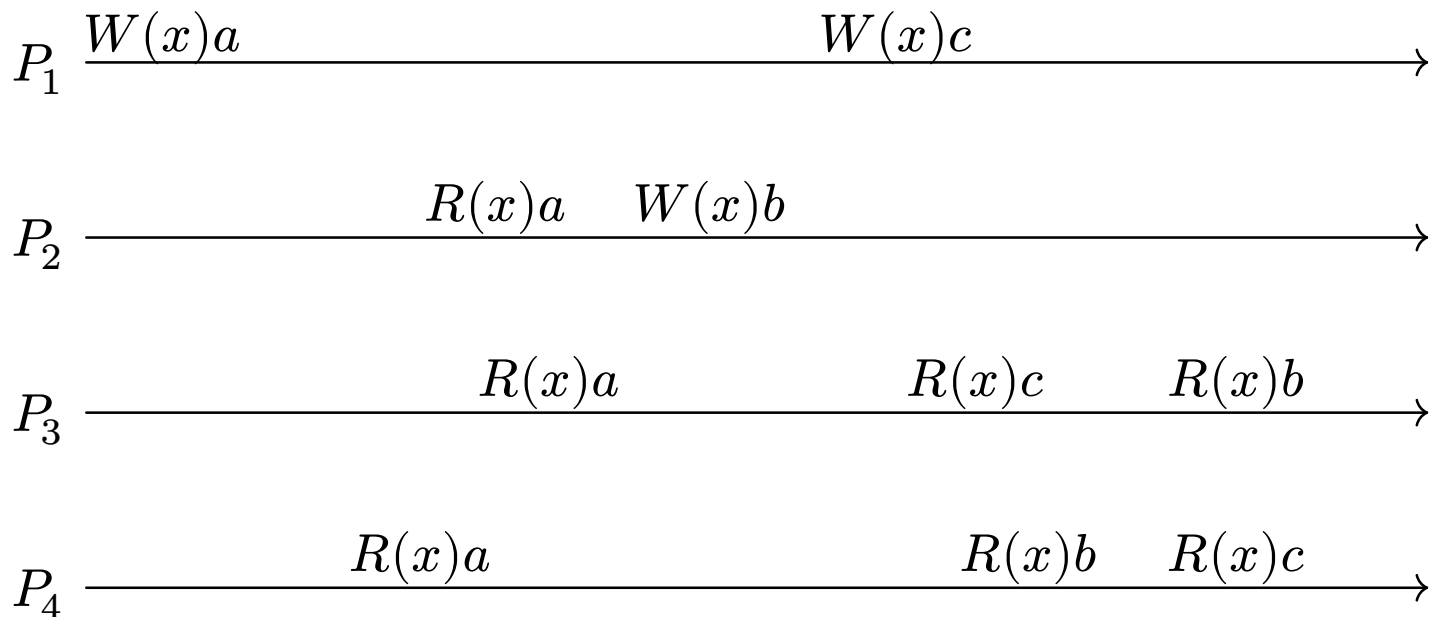
A stronger form of sequential consistency where each operation appears to take effect instantaneously at some moment between its start and completion.

- **Temporal Aspect:** Unlike sequential consistency, linearizability requires that if operation A completes before operation B starts, A must precede B in the sequential order.
- **Compositionality:** Linearizability is compositional; if individual data items are linearizable, their set is also linearizable.
- **Cost:** Implementing linearizability on many-core or distributed architectures poses serious performance challenges due to the need for strict synchronization.

Causal Consistency

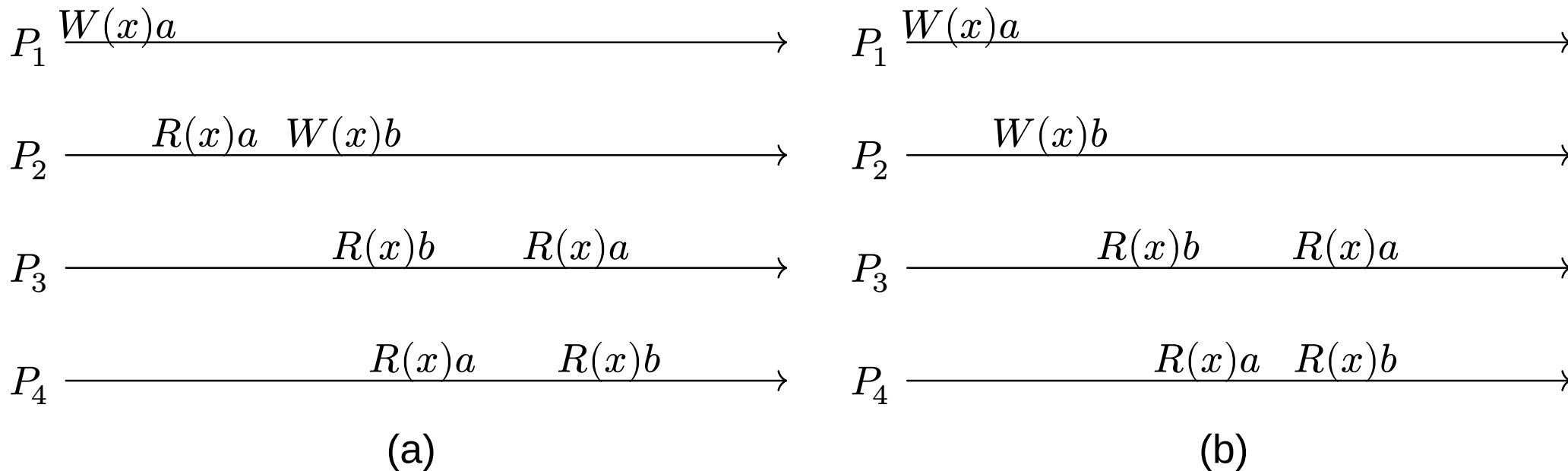
- **Principle:** Writes that are potentially causally related must be seen by all processes in the same order.
- **Causal Relation:** If process P_2 reads a value written by P_1 and then performs a write, the two writes are causally related.
- **Concurrent Writes:** Writes that are not causally related (spontaneous/simultaneous) may be seen in different orders on different machines.
- **Implementation:** Requires maintaining a dependency graph of operations, often implemented using metadata like vector timestamps.

Causal Consistency: Example



Causally consistent **but not** sequentially consistent – $W_1(x)c$ and $W_2(x)b$ are concurrent

Causal Consistency: Example



NOT causally consistent

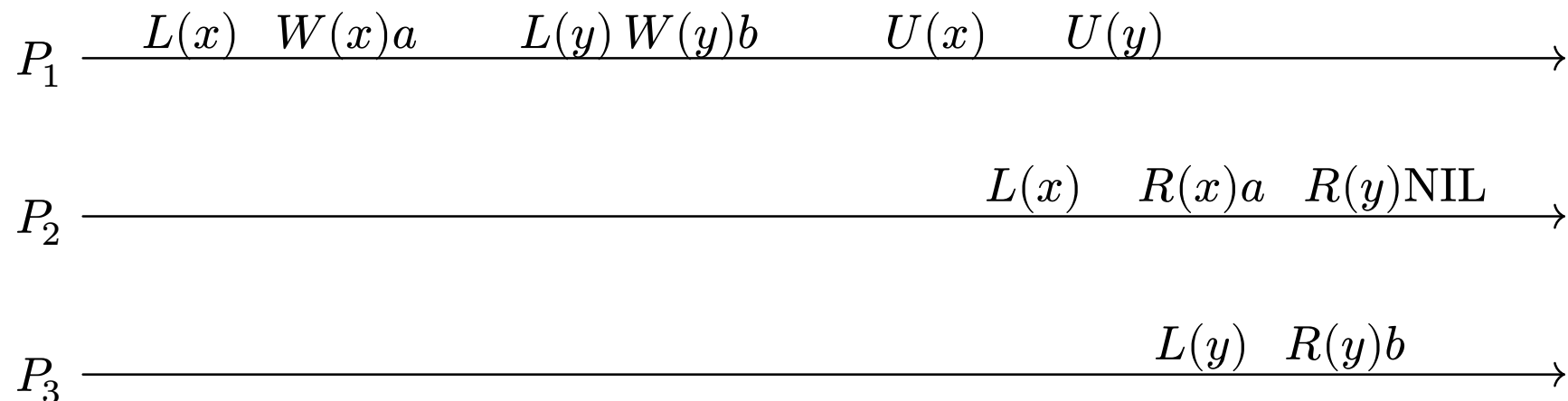
causally consistent

Entry Consistency (Grouping Operations)

Instead of consistent ordering at the level of individual read/write operations, operations are grouped and protected by synchronization variables (locks).

- **Acquisition Rule:** Acquiring a lock can succeed only when all updates to its associated shared data have completed
- **Exclusive Access:** Exclusive access to a lock (for writing) is granted only if no other process has exclusive or non-exclusive access
- **Stale Reads:** A simple read operation performed without first acquiring the relevant lock may result in reading a local value that is stale

Entry Consistency (Grouping Operations): Example



- **Notation:**

- $L(x)$ denotes a lock on x
- $U(x)$ denotes unlock of x

P_2 does not have a lock on y , therefore, it can read NIL

Eventual Consistency

If no updates take place for a long time, all replicas will eventually converge toward identical copies of each other

- **Characteristics:**

- Requires only that updates (writes) are guaranteed to propagate to all replicas
- Write-write conflicts are assumed to be rare or easily resolvable
 - e.g., “last writer wins”

- **Use Cases:** Ideal for large-scale systems where most operations are reads and inconsistencies are tolerable, such as DNS and Web caches

- **Scalability:** Cheap to implement as it avoids expensive global synchronization

Continuous Consistency

- **Goal:** To bound the degree of inconsistency between replicas along 3 specific dimensions
- **Dimensions of Deviation:**
 1. **Numerical Deviation:** The difference in numerical value between a local replica and the “actual” value if all updates were applied
 2. **Staleness Deviation:** The time elapsed since the last update was performed at a remote origin
 3. **Ordering Deviation:** The number of tentative (uncommitted) updates that have been applied locally but not yet globally ordered
- **Bound Enforcement:** Replicas must synchronize when local deviations exceed a predefined threshold δ

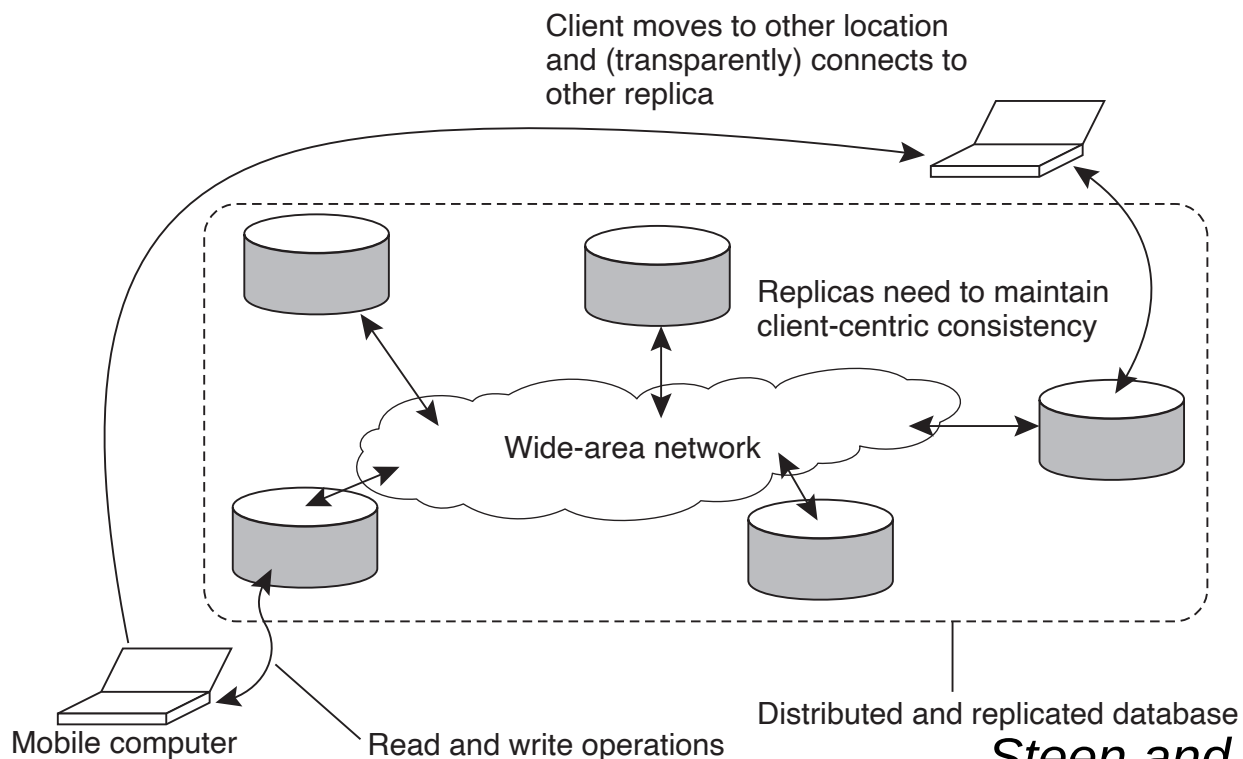
Client-Centric Consistency

Client-Centric Consistency

Data-centric models focus on system-wide consistency, but maintaining this is often too expensive for large-scale systems.

- **Focus:** Client-centric models concentrate on consistency from the perspective of a **single client** (often mobile) interacting with different replicas.
- **Context:** These models are typically used in data stores that offer **eventual consistency**, where replicas converge over time but may be temporarily out of sync.
- **Key Benefit:** They hide inconsistencies from a specific user even when that user connects to different, potentially stale replicas.

Client-Centric Consistency



Steen and Tanenbaum (2023)

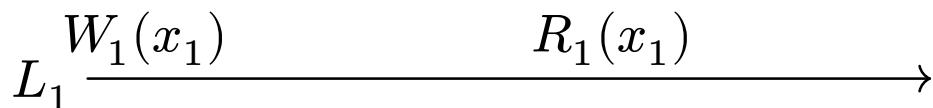
Monotonic Reads

If a process reads the value of a data item x , any successive read operation on x by that same process will always return that same value or a more recent value.

- **Guarantee:** Once a process has seen a specific version of a data item, it will never see an older version of that same item.
- **Example:** In a distributed email database, if a user reads their mailbox in San Francisco and then opens it in New York, they are guaranteed to see at least the emails they already saw in San Francisco.

Monotonic Reads: Example

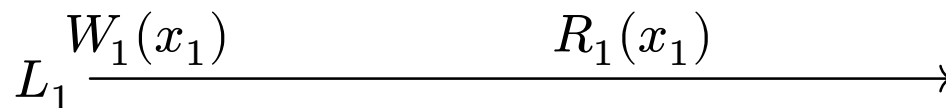
We know operate on *local data stores* L_i



(a)

monotonic-read consistent

$W_2(x_1; x_2)$: x_2 is created after x_1



(b)

NOT monotonic-read consistent

$W_2(x_1 | x_2)$: x_2 is concurrent to x_1

Monotonic Writes

A write operation by a process on a data item x is completed before any successive write operation on x by the same process.

- **Ordering Requirement:** A new write is performed only on a replica that has been brought up to date with all preceding writes by that same process.
- **Example:** When updating a software library, this model ensures that if an update replaces a function, all previous updates that were part of that library's development history are applied first.

Monotonic Writes: Examples

$L_1 \xrightarrow{W_1(x_1)}$

$L_2 \xrightarrow{W_2(x_1; x_2) \quad W_1(x_2; x_3)}$

(a) consistent

$L_1 \xrightarrow{W_1(x_1)}$

$L_2 \xrightarrow{W_2(x_1 | x_2) \quad W_1(x_1 | x_3)}$

(b) **NOT** consistent

$L_1 \xrightarrow{W_1(x_1)}$

$L_2 \xrightarrow{W_2(x_1 | x_2) \quad W_1(x_2; x_3)}$

(c) **NOT** consistent

$L_1 \xrightarrow{W_1(x_1)}$

$L_2 \xrightarrow{W_2(x_1 | x_2) \quad W_1(x_1; x_3)}$

(d) consistent

Read Your Writes

The effect of a write operation by a process on data item x will always be seen by a successive read operation on x by the same process.

- **User Experience:** A process will never read a value that does not include its own most recent update.
- **Example:** If a user updates a Web document (e.g., through an editor) and immediately views it in a browser, the browser must not return a cached, stale version that lacks the edit.
- **Example:** Password updates; a user expects their new password to work immediately after they have changed it.

Read Your Writes: Example

$L_1 \xrightarrow{W_1(x_1)}$

$L_2 \xrightarrow{W_2(x_1; x_2) \quad R_1(x_2)}$

(a) consistent

$L_1 \xrightarrow{W_1(x_1)}$

$L_2 \xrightarrow{W_2(x_1 | x_2) \quad R_1(x_2)}$

(b) **NOT** consistent

Write Follows Reads

A write operation by a process on a data item x following a previous read operation on x by the same process is guaranteed to take place on the same or a more recent value of x that was read.

- **Causal Link:** Ensures that a write operation is performed only on a copy that is up to date with the value most recently read.
- **Example:** In a network newsgroup, this model guarantees that a user's reaction (write) to an article is only stored at a local copy if the original article (read) is also stored there.

Write Follows Reads: Example

L_1 $\xrightarrow{W_1(x_1) \quad R_2(x_1)}$

L_2 $\xrightarrow{W_3(x_1; x_2) \quad W_2(x_2; x_3)}$

(a) consistent

L_1 $\xrightarrow{W_1(x_1) \quad R_2(x_1)}$

L_2 $\xrightarrow{W_3(x_1 | x_2) \quad W_2(x_2 | x_3)}$

(b) **NOT** consistent

Replica Management

Replica Management Fundamentals

- **Core Objective:** Decide where, when, and by whom replicas should be placed, and how updates are subsequently distributed among them
- **Key Components:**
 - **Replica Placement:** Finding the optimal locations for placing replicas to enhance performance and reliability
 - **Update Propagation:** Ensuring that all replicas eventually reach the same state after an update
- **Consistency Requirement:** The management strategy must support the specific consistency model required by the application

Find The Best Server Location

Replica-server placement is generally treated as finding the best K out of N possible locations ($K < N$).

- **Complexity:** This is a computationally complex problem that typically requires the use of heuristics rather than exhaustive search
- **Criteria for “Best”:**
 - **Cost-related:** Focusing on administrative or monetary expenses.
 - **Network-related:** Focusing on latencies, bandwidth, logical distance, or hop count.
- **Modern Context:** Precise location has become less critical due to the ubiquity of large-scale data centers and improved global connectivity

Find The Best Server Location: Algorithms

Placement algorithms are categorized into several key classes:

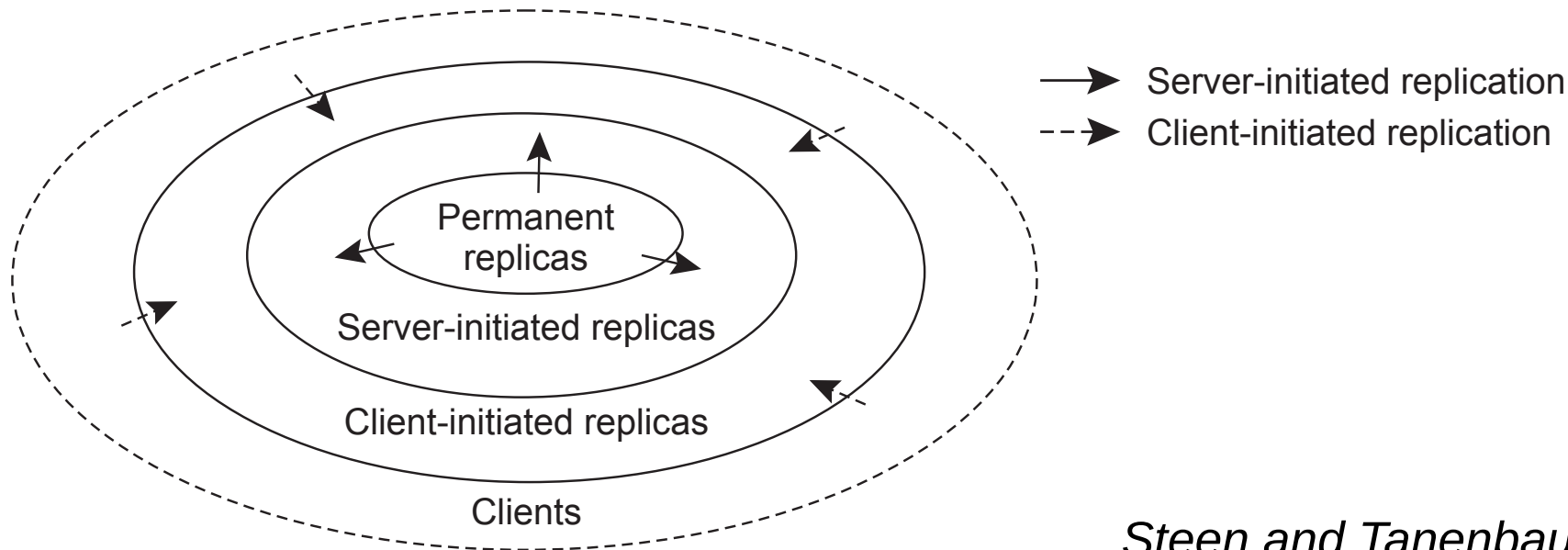
- **Quality of Service (QoS) Aware:** Optimized for parameters like bandwidth or latency; may seek “bounded” guarantees (e.g., minimum guaranteed bandwidth).
- **Consistency Aware:** Focuses on minimizing the cost of keeping content up to date based on update propagation methods (periodic, aperiodic, expiration-based).
- **Energy Aware:** Places servers to minimize energy consumption, often by considering power modes or workload distribution.
- **Others (Monetary/Policy):** Especially in CDNs, placement may involve negotiating monetary costs with different Autonomous Systems (AS).

Findind The Best Server Location: Challenges

- **Data Imprecision:** Measuring exact parameters like latency between a server and an end-user client is inherently difficult because clients cannot always be directly accessed for testing.
- **Heuristics vs. Theory:** While many theoretical models exist, they often rely on having perfect knowledge of future read/write patterns, which is rarely available.
- **Practicality:** Because input values (like bandwidth and usage patterns) are often imprecise or unpredictable, “quick-and-dirty” heuristics are frequently more practical than overly complex theoretical optimizations.

Content Replication and Placement

Replicas are generally categorized into three types based on who initiates the replication: **Permanent**, **Server-initiated**, and **Client-initiated**



Steen and Tanenbaum (2023)

Permanent Replicas

The initial set of replicas that constitute the distributed data store.

- **Characteristics:**
 - Typically small in number
 - Often manually configured and static
- **Example (Mirroring):** A Web site where a limited number of “mirror sites” are set up at specific, well-known locations to share the load of the original server
- **Shared-Nothing Architecture:** Permanent replication is managed within a “shared-nothing” architecture to ensure that the failure of one node does not impact others
 - neither disks nor main memory are shared by processors

Server-Initiated Replicas

Replicas created dynamically by the owner of a data store to enhance performance in response to changing client demands.

- **Purpose:** To “push” content closer to clients that frequently access specific data, thereby reducing latency and balancing the workload across servers
- **Mechanism:**
 - The system monitors access patterns for individual data items.
 - If a server S detects high demand for item x from a remote region, it may decide to install a copy of x on a server closer to those clients
- **Usage:** Commonly used in Content Delivery Networks (CDNs) to dynamically place popular videos or images on edge servers

Client-Initiated Replicas

Temporary copies of data created by clients to improve their own access times. Unlike replication, caching is a decision made by the **client** (the consumer), not the owner of the resource.

- **Storage:** Data is usually kept in a local cache
 - e.g., a Web browser's cache or a mobile device's local storage
- **Management:**
 - The client is responsible for managing the cache
 - e.g., deciding when to delete old items
 - **Consistency Problem:** Caches may return stale data if the original resource is updated without the cache being invalidated or refreshed

Content Distribution

Content Distribution Fundamentals

The process of propagating updates from the replica where the update was initiated to all other replicas in the system.

- **Core Decision:** Designers must choose **what** is propagated, **how** it is propagated (push vs. pull), and the **protocol** used to ensure consistency.
- **Performance Impact:** The choice of distribution strategy significantly affects the bandwidth consumption, latency, and the staleness of data at various replicas.

What is Propagated

- **Notifications:** Only an invalidation message is sent, informing other replicas that their data is now stale
 - **Pros:** Very low bandwidth usage
 - **Cons:** Requires a subsequent fetch of the actual data when needed
- **Data Transfers:** The modified data itself is sent
 - **Pros:** Replicas are updated immediately; good for high read-to-update ratios
 - **Cons:** High bandwidth consumption if updates are frequent
- **Operations (Active Replication):** Only the update command (e.g., an RPC call) is sent to all replicas.
 - **Pros:** Minimal bandwidth usage
 - **Cons:** Requires more processing power at each replica to re-execute the operation

Push-based (Server-Initiated) Propagation

- **Mechanism:** Updates are pushed to other replicas without those replicas asking for them
- **Consistency:** Often used to maintain high consistency
 - e.g., sequential or causal consistency
- **Efficiency:** Best suited for systems with a high read-to-update ratio, as it ensures readers always find fresh data
- **Overhead:** The server must keep track of all replicas/clients that need updates

Pull-based (Client-Initiated) Propagation

- **Mechanism:** A replica or client-side cache polls a server to see if its local data is still valid
- **Consistency:** Results in lower consistency; data remains stale until the next poll
- **Efficiency:** Best for systems with a low read-to-update ratio (avoiding unnecessary pushes for data that won't be read)
- **Overhead:** No state needs to be maintained at the server regarding which clients have which data

Hybrid: Leases

A contract where the server promises to push updates to a client for a specific period (the lease).

- **Behavior:**
 - While the lease is valid, the system behaves as **Push-based**
 - Once the lease expires, the client must request a new lease or poll the server (**Pull-based**)
- **Dynamic Leases:** The duration of a lease can be adjusted based on the observed popularity of the data or the update frequency
- **Benefit:** Combines the consistency of push with the state-management efficiency of pull

Gossip Protocols (Epidemic Algorithms)

- **Principle:** Based on the theory of how diseases spread in a population
- **Anti-entropy:** Replicas periodically choose another replica at random and exchange data to resolve inconsistencies
- **Rumor Spreading:** When a replica receives an update, it tries to “infect” other replicas by pushing the update to them
- **Convergence:** Extremely effective for spreading updates in large-scale, eventual-consistency systems like Amazon’s Dynamo or Apache Cassandra

Managing Replicated Objects

Managing Replicated Objects

Core Problem: How to apply replication techniques to objects that encapsulate both data (state) and the operations to manipulate that data.

- **Approach:** When an object is replicated, we must decide whether to replicate the **state** of the object or the **operations** that change the state.
- **Consistency Requirement:** In both cases, the goal is to ensure that all replicas reach a consistent state, typically through some form of atomic broadcast or coordinated execution.

Primary-Backup Protocols (Passive Replication)

One replica is designated as the **primary**, while others act as **backups**.

- **Process:**
 1. The client sends a request to the primary
 2. The primary executes the operation locally
 3. The primary sends the updated **state** to all backups
 4. Once backups acknowledge, the primary responds to the client
- **Benefit:** Simple to implement and handles non-deterministic operations easily (since the primary decides the outcome)
- **Drawback:** The primary is a potential performance bottleneck and a single point of failure

Active Replication (State Machine Replication)

Every replica is a state machine that executes the same sequence of operations.

- **Process:**
 1. Requests are sent to all replicas using **Totally Ordered Multicast** (Atomic Broadcast)
 2. Every replica executes the operation in the same order
 3. Because they start in the same state and execute the same operations in the same order, they remain synchronized
- **Strict Requirement:** Operations must be **deterministic**; if the same operation is executed on two identical states, it must produce the same result

Active Replication vs. Passive Replication

Feature	Passive (Primary-Backup)	Active (State Machine)
What is sent	State updates	Operations/Invocations
Determinism	Not required	Mandatory
Failure recovery	Slow (new primary needed)	Fast (others already running)
Network overhead	High (if state is large)	Low (small commands)

Quorum-Based Protocols for Objects

Instead of a single primary, a majority of replicas must agree before an operation is finalized.

- **Read and Write Quorums:**
 - To read an object, a client must contact N_R replicas
 - To write (update) an object, a client must contact N_W replicas
- **Condition:** $N_R + N_W > N$ (where N is the total number of replicas)
- **Effect:** This ensures that any read quorum contains at least one replica from the most recent write quorum, guaranteeing that the most up-to-date version is found

Resources

Resources

- M. van Steen and A.S. Tanenbaum, Distributed Systems, 4th ed., distributed-systems.net, 2023.